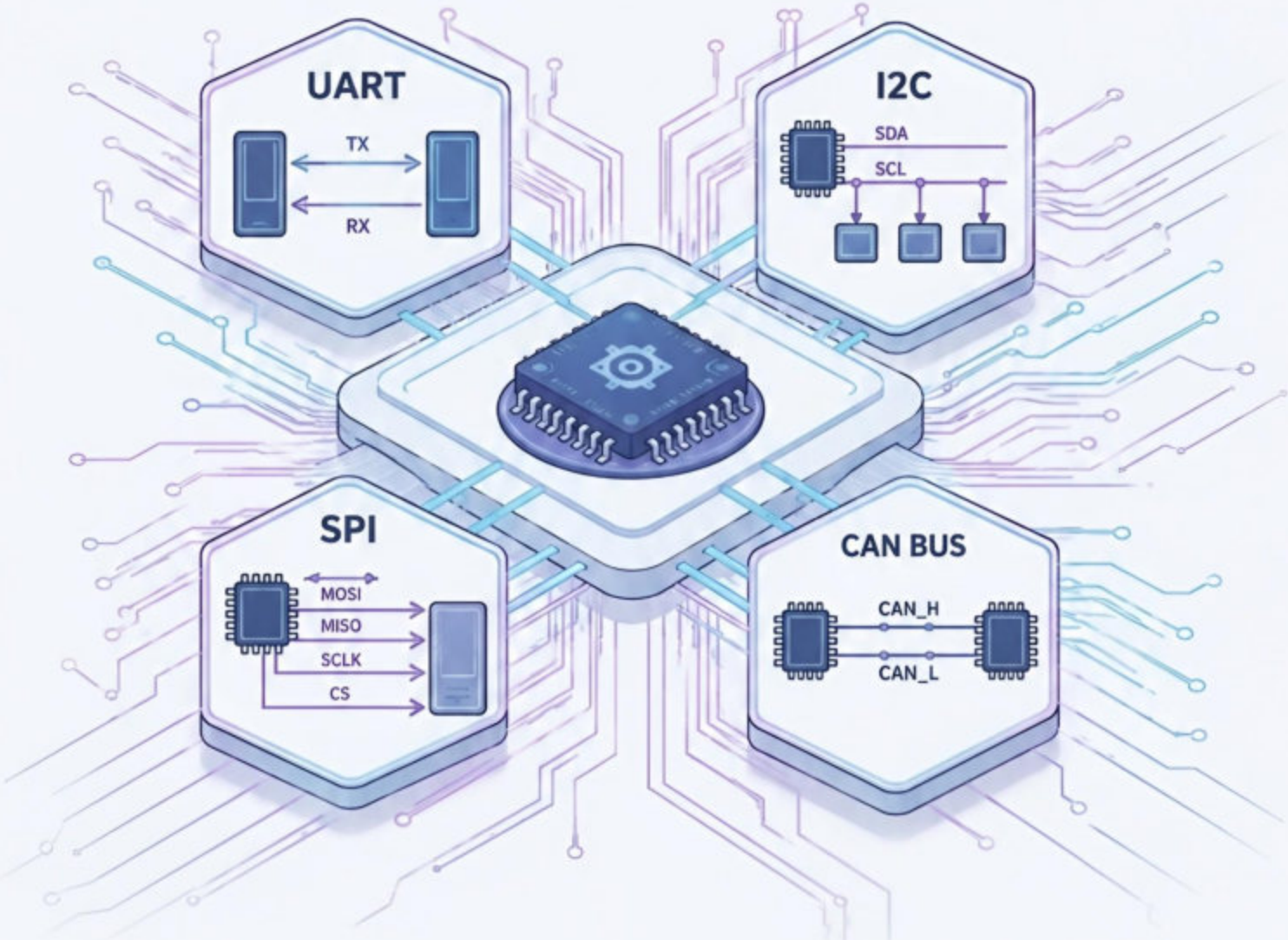
فريق نون التطوعي - السنة الرابعة

المعالجات الصغرية و نظمها

الملحق الرابع :

بروتوكولات الاتصال

قسم د.ماهر علاوي



كتابة وتدقيق: بلال السابق

تفريغ: سنا ذوالغنى - ليلاس إدلبي

قسم هندسة الالكترونيات والاتصالات

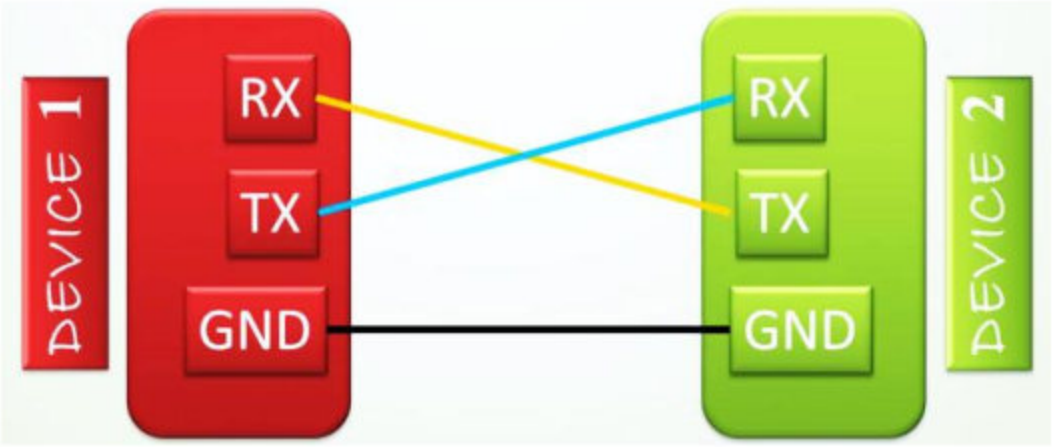
2025 - 2026

بروتوكولات الاتصال

بروتوكولات الاتصال التسلسلية :

أولاً: بروتوكول UART (UART : Universal Asynchronous Reception and Transmission):

- وتعني الاستقبال و الإرسال الشامل غير المتزامن ..



- وهو بروتوكول اتصال تسلسلي بسيط يسمح لجهازين بالتواصل فيما بينهما

- يدعم النقل ثنائي الاتجاه

- يستخدم خطين لنقل البيانات ، Tx للإرسال ، و Rx للاستقبال

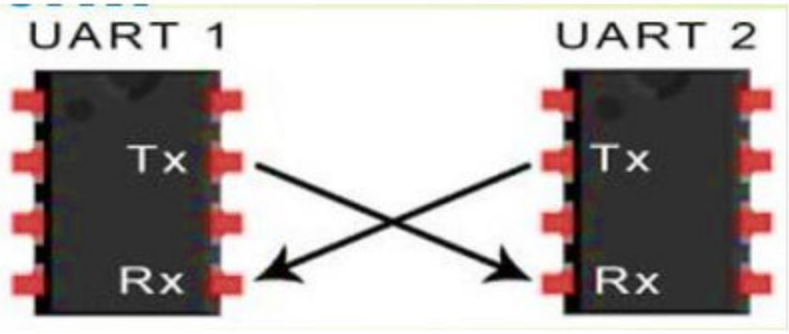
- يمكنه نقل البيانات بين الأجهزة بثلاث طرق :

1. Simplex : لنقل البيانات باتجاه واحد .
2. Half – Duplex : لنقل البيانات في أي اتجاه لكن ليس بنفس الوقت .
3. Full – Duplex : لنقل البيانات في كلا الاتجاهين و بنفس الوقت .

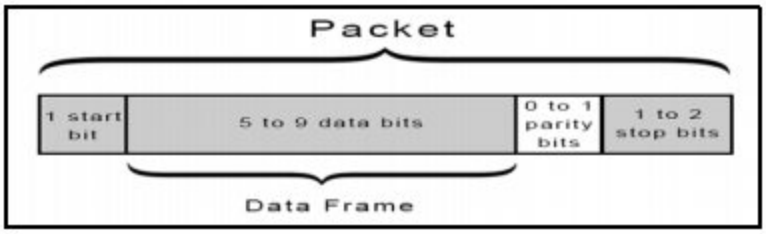
- بما أن بروتوكول UART غير متزامن فإنه لا يحتاج لنبضات ساعة .

- عند إرسال بيانات عبر واجهة UART من الجهاز الرئيسي (مثل CPU) ، يتم تحويل البيانات المتوازية Parallel إلى شكل تسلسلي Serial و يرسلها إلى واجهة UART في المستقبل ، ثم يقوم المستقبل بتحويل البيانات التسلسلية إلى متوازية مرة أخرى .

كيف يعمل بروتوكول UART ؟



1. يقوم مرسل UART أولاً باستقبال البيانات من ناقل البيانات (Bus) الذي يرسله مكون آخر مثل المعالج CPU .
2. بعد استلام البيانات من ناقل البيانات يقوم بإضافة ثلاثة بتات هي : بت البداية ، و بت التحقق (Parity) ، و بت الإيقاف ، وذلك لتكوين حزمة البيانات .



3. ثم يتم إرسال حزمة البيانات عبر طرف الإرسال Tx ، حيث يقوم UART المستقبل بقراءة هذه الحزمة عبر طرف الاستقبال الخاص به Rx .
4. يستمر إرسال البيانات طالما توجد بيانات متبقية داخل وحدة UART المرسلة .

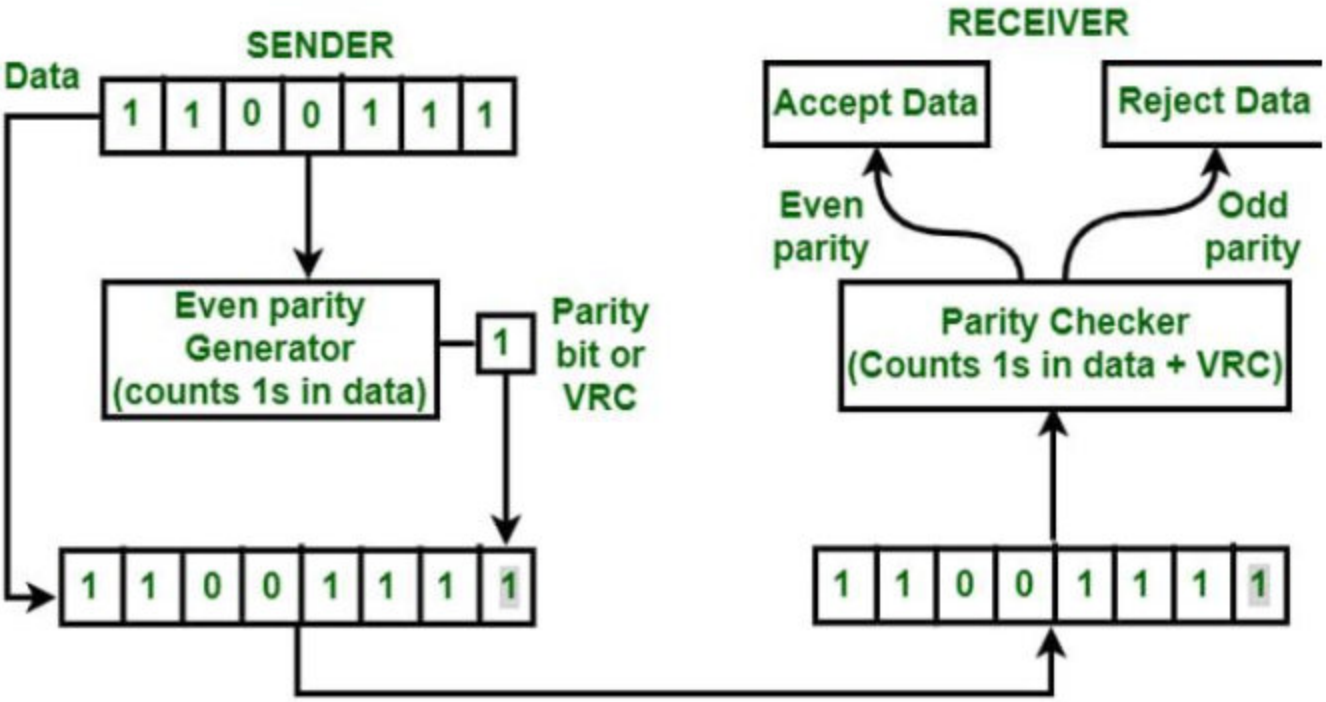
بت التحقق Parity :

هو وسيلة لجهاز UART المستقبل لمعرفة ما إذا كان أي من البيانات قد تغير أثناء الإرسال، بعد أن يقرأ UART المستقبل إطار البيانات يقوم بإحصاء عدد البتات ذات القيمة 1 و يتحقق مما إذا كان عدداً زوجياً أم فردياً .

7 bits of data	(count of 1-bits)	8 bits including parity	
		even	odd
0000000	0	00000000	00000001
1010001	3	10100011	10100010
1101001	4	11010010	11010011
1111111	7	11111111	11111110

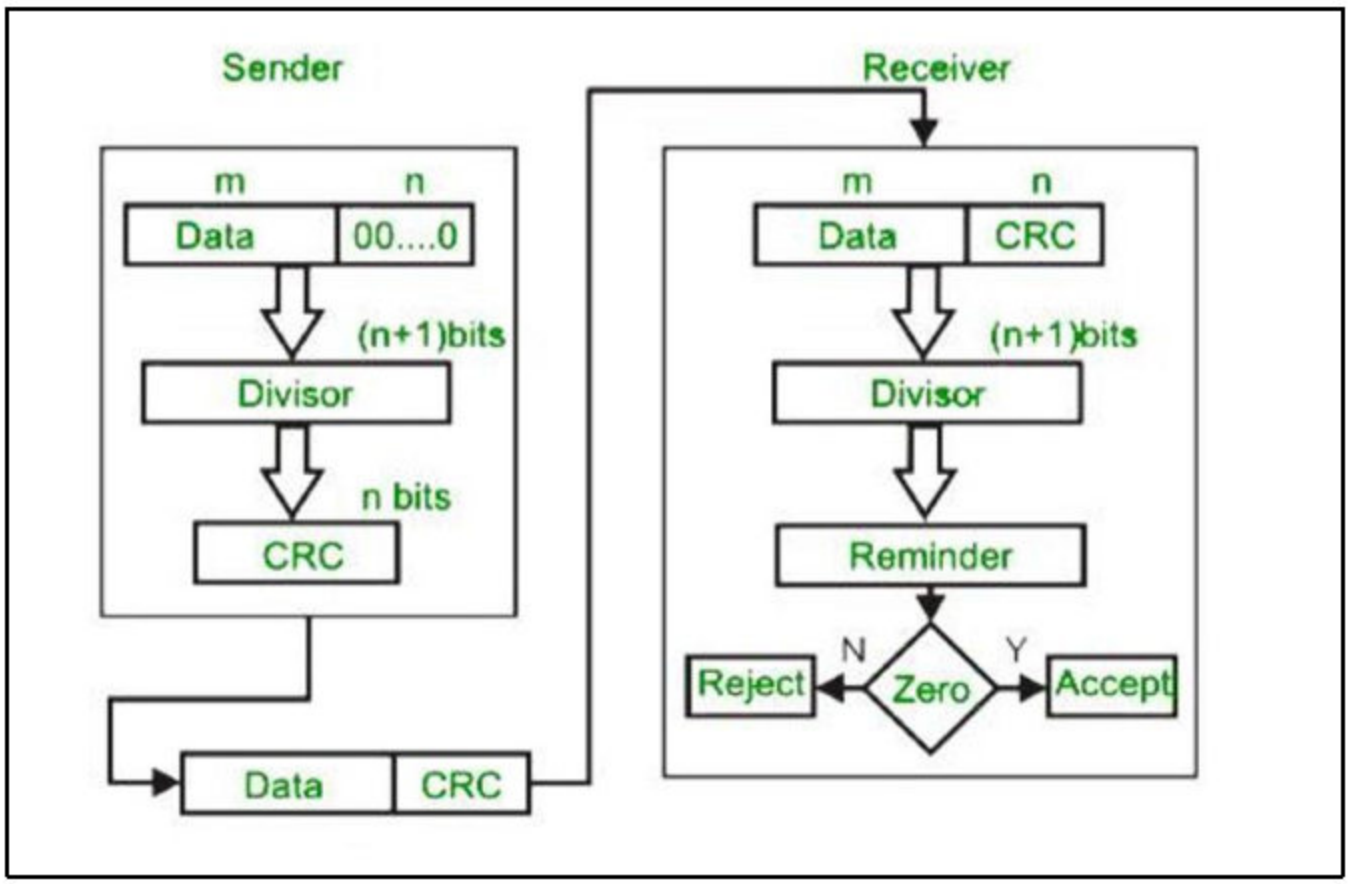
يمكن إجراء الاختبار للتماثل الزوجي أو الفردي (بحسب ما يتفق عليه المرسل و المستقبل)

مثال للحالة الزوجية _ even case :



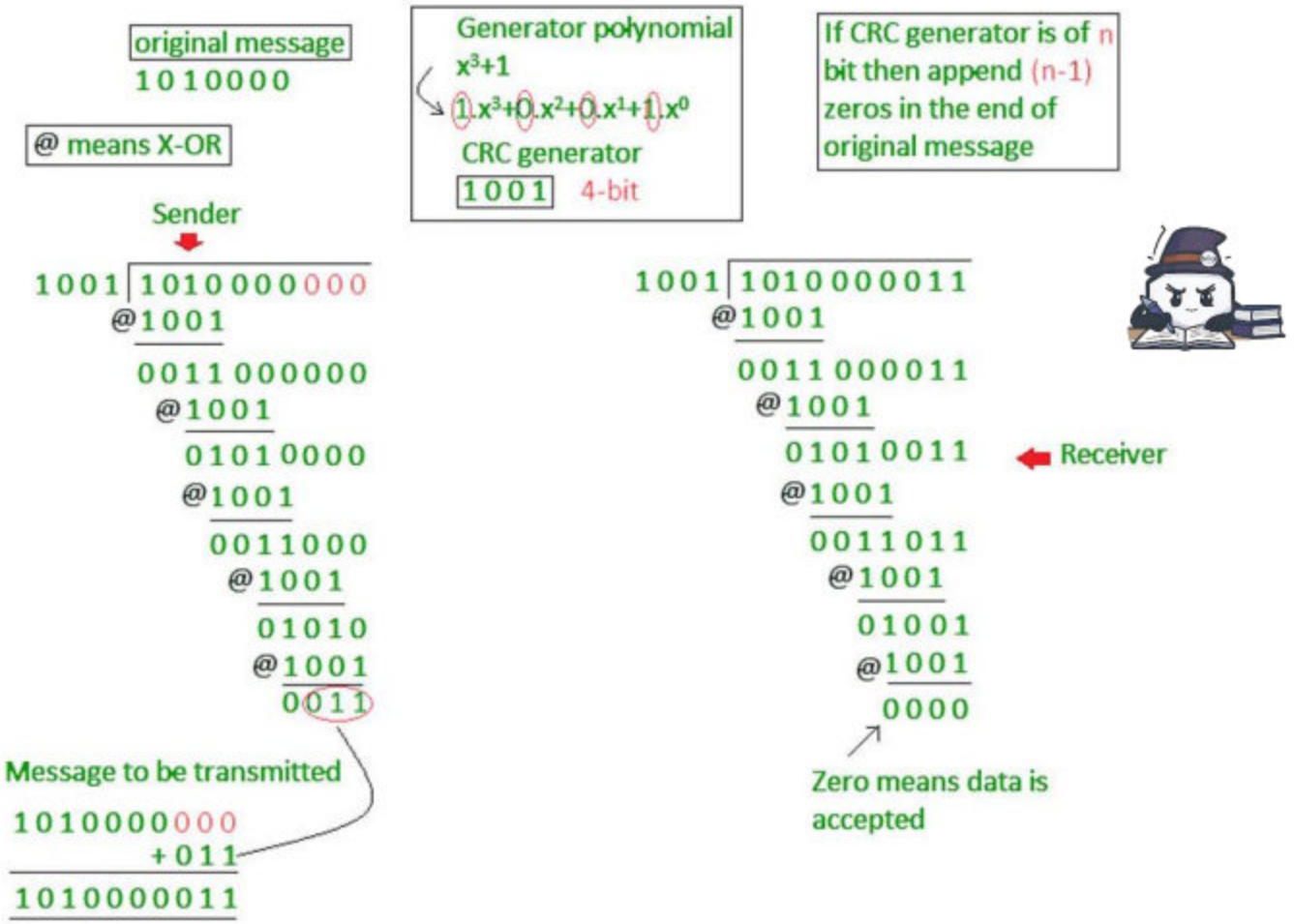
فحص التكرار الدوري CRC :

إن طريقة Parity bit طريقة بدائية لذلك تُستخدم أحياناً طريقة CRC لاكتشاف الأخطاء في قناة الاتصال ، ويستخدم كلٌّ من المرسل و المستقبل المفتاح ذاته ..



شرح الخوارزمية السابقة :

- ✓ البيانات لدينا مكونة من m-bit ، نضيف إليها n-bit من الأصفار .
- ✓ نقسم الحزمة الناتجة لدينا على عدد معين متفق عليه بين المرسل والمستقبل ومكون من (n+1)-bit
- ✓ نأخذ باقي القسمة المكون من n-bit (ويكون هو CRC) ونضيفه إلى الData الأصلية ، وتكون هذه الرسالة النهائية المرسلة عبر قناة الاتصال .
- ✓ في قسم الاستقبال يتم تقسيم هذه الرسالة المستلمة على نفس العدد الذي تم التقسيم عليه في المرسل ، ونأخذ باقي القسمة ، فإذا كان صفراً فهذا يعني أن الرسالة صحيحة وخالية من الأخطاء ، وإلا فإن الرسالة قد تعرضت لضجيج .
- ✓ مثال على ذلك :



مزايا استخدام UART :

1. سهلة الاستخدام ولها الكثير من المصادر على الانترنت .
2. لا تحتاج نبضات تزامن .
3. تدعم Parity bit لفحص الأخطاء .

مساوئ استخدام UART :

1. حجم إطار البيانات بحدود 9 bit فقط .
2. لا تدعم تعدد الأجهزة وإنما مرسل واحد ومستقبل واحد فقط .
3. يجب أن تكون Boad rate في كل جهاز UART متقاربة ضمن حدود 10% من بعضها ، وذلك لتجنب فقدان البيانات .
4. سرعة نقل البيانات منخفضة .

أمثلة على أجهزة تستخدم بروتوكول UART :

- وحدة البلوتوث HC-05
- وحدة GPS

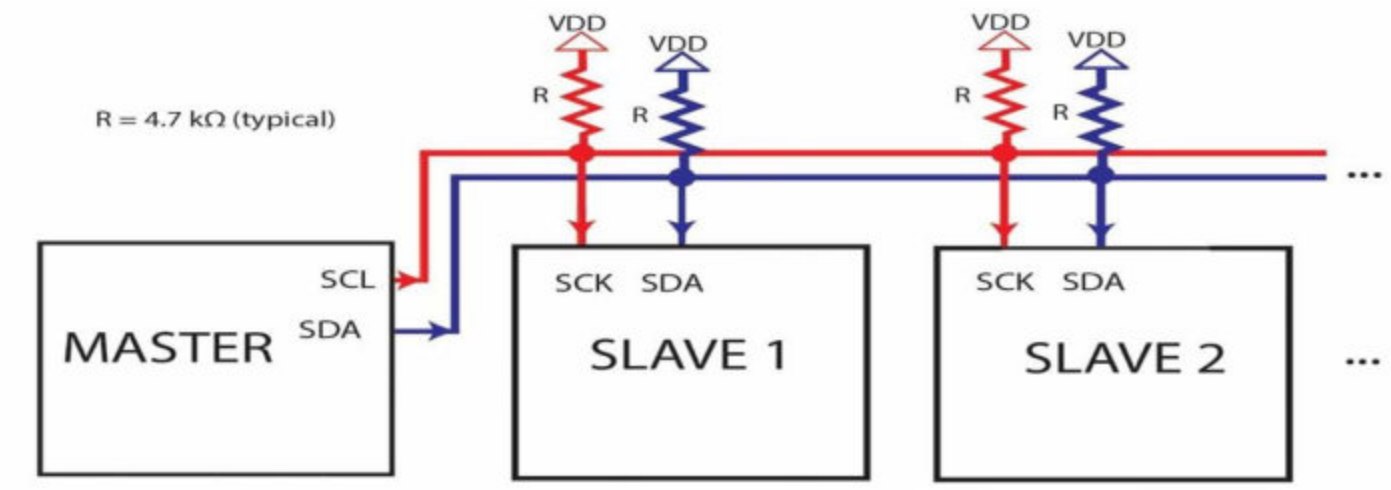
- هناك أيضاً بروتوكول شبيه بـ UART يُدعى USART ، والفرق الجوهري بينهما هو أن بروتوكول USART متزامن ، بالإضافة إلى عدد من الفروقات الأخرى الموضحة في الجدول أدناه :

	UART	USART
Name Abbreviation	**U**niversal **A**synchronous **R**eception and **T**ransmission	**U**niversal **S**ynchronous/**A**synchronous **R**eception and **T**ransmission
Mode	Full duplex	Half duplex
Signal Count	**2** (Tx and Rx)	**4** (Tx, Rx, XCK (clock), and XDIR (direction))
Data Rate	Specified between transmitter and receiver	Defined by the clock pulse stream on the XCK pin
Complexity	Less complex circuitry and fewer communication lines	More complex circuitry and more communication lines

بروتوكول I2C :

- يرمز الاسم إلى Inter Integrated Circuit
- وهو بروتوكول اتصال تسلسلي يستخدم مع الModules و الSensors
- يستخدم سلكين فقط لنقل المعلومات ، وهو بروتوكول ثنائي الاتجاه ومتزامن .
- يدعم توصيل ما يصل إلى 128 جهاز باللوحة الرئيسية .
- يستخدم نظام العناوين و bus مشترك ، أي يمكن توصيل العديد من الأجهزة المختلفة باستخدام نفس الأسلاك بحيث يكون لكل جهاز عنوان مختلف عن الآخر
- هذا البروتوكول أبطأ من بروتوكول SPI
- سرعة I2C تعتمد على سرعة البيانات و جودة الأسلاك والضوضاء الخارجية
- العديد من وحدات وحساسات أردوينو تدعم I2C

كيفية توصيل الأجهزة ببعضها عبر بروتوكول I2C :



كيف يعمل بروتوكول I2C ؟

1- يتكون بروتوكول I2C من خطّين رئيسيين هما:

SCL (Serial Clock Line) خط الساعة التسلسلية

و SDA (Serial Data Line) خط البيانات التسلسلية.

2- يعمل خط SCL على توليد نبضات الساعة اللازمة لمزامنة عملية إرسال واستقبال البيانات بين الأجهزة، بينما يستخدم خط SDA لنقل واستقبال بتات البيانات.

3- يقوم الجهاز الرئيسي Master ببدء عملية الاتصال على الناقل (Bus)، كما أنه المسؤول عن توليد إشارة الساعة. أما أي جهاز يتم تحديد عنوانه أثناء الاتصال فيُعتبر جهازًا تابعًا Slave.

4- عندما يريد الـ Master إرسال بيانات إلى جهاز تابع Slave، يجب عليه أولًا إرسال عنوان الجهاز المطلوب قبل البدء بإرسال البيانات.

5- بعد الانتهاء من إرسال البيانات، يقوم الـ Master بإنهاء عملية الاتصال.

6- إذا أراد الـ Master استقبال بيانات من جهاز تابع Slave، فإنه يجب عليه أيضًا أولًا تحديد عنوان الجهاز المطلوب.

7- بعد ذلك يستقبل الـ Master البيانات المرسلة من الـ Slave، ثم يقوم بإنهاء عملية الاستقبال. كما يبقى الـ Master مسؤولًا عن توليد إشارة التوقيت وإنهاء نقل البيانات.

8- يجب توصيل خطوط الاتصال بمصدر التغذية عبر مقاومات رفع (Pull-up Resistors)، وذلك لأن خطي SCL و SDA يكونان في حالة مستوى منطقي مرتفع High عند عدم وجود عملية اتصال على الناقل.

———————————————————————

ميزات بروتوكول I2C :

- ✓ يتميز بروتوكول I2Cابعدد قليل من خطوط التوصيل، حيث يمكن ربط عدد كبير من الأجهزة على نفس الناقل باستخدام عدد محدود من الأرجل .
- ✓ يتميز بالمرونة، لأنه يدعم الاتصال بين عدة أجهزة رئيسية Multi-Master وعدة أجهزة تابعة Multi-Slave على نفس الناقل.
- ✓ يعتبر بسيطًا في التوصيل، لأنه يعتمد فقط على خطين ثنائيي الاتجاه SCL و SDA لإنشاء الاتصال بين عدة أجهزة.
- ✓ يتميز بالقدرة على التكيف، حيث يمكنه التعامل مع احتياجات وأنماط مختلفة من الأجهزة التابعة Slave Devices.
- ✓ يدعم وجود أكثر من جهاز رئيسي Master، بحيث يمكن لأي جهاز رئيسي التحكم بعملية الاتصال على الناقل حسب الحاجة.

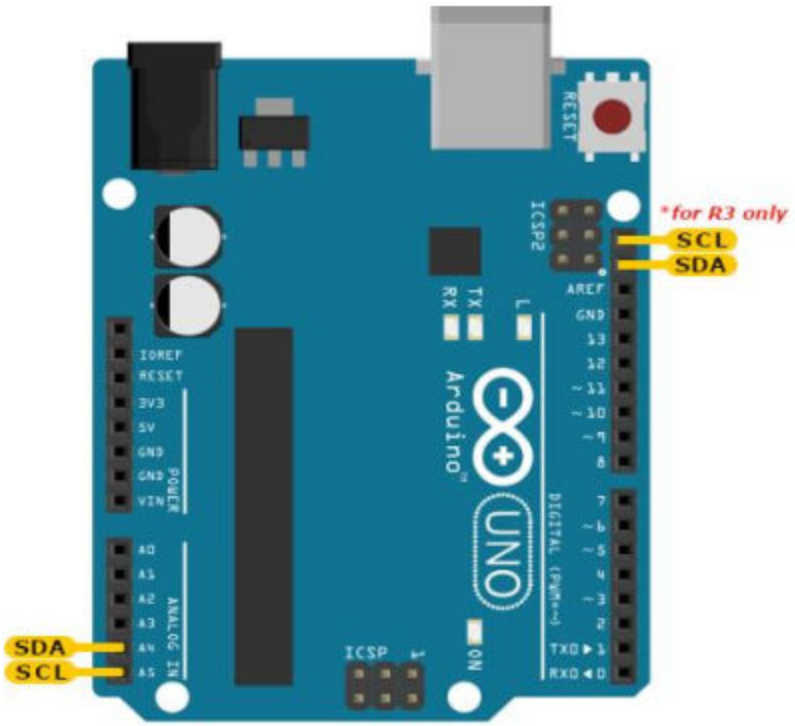
مساوئ بروتوكول I2C :

- تكون سرعة الاتصال أقل مقارنة ببعض البروتوكولات مثل SPI، وذلك لأن بروتوكول I2C يعتمد على مقاومات الرفع Pull-up Resistors بدلًا من دوائر الدفع المباشر Push-pull المستخدمة في SPI. كما أن تصميمه يعتمد على خرج Open-drain مما يحد من سرعة نقل البيانات.
- يحتاج إلى مساحة إضافية على لوحة الدارة المطبوعة PCB بسبب وجود مقاومات الرفع، مما قد يشغل مساحة مهمة من اللوحة.
- قد تصبح عملية إدارة الاتصال أكثر تعقيدًا عند زيادة عدد الأجهزة المتصلة على نفس ناقل I2C.

الأرجل الخاصة ببروتوكول I2C في لوحة Arduino UNO ؟

- المنفذ A4 يمثل SDA
- المنفذ A5 يمثل SCL

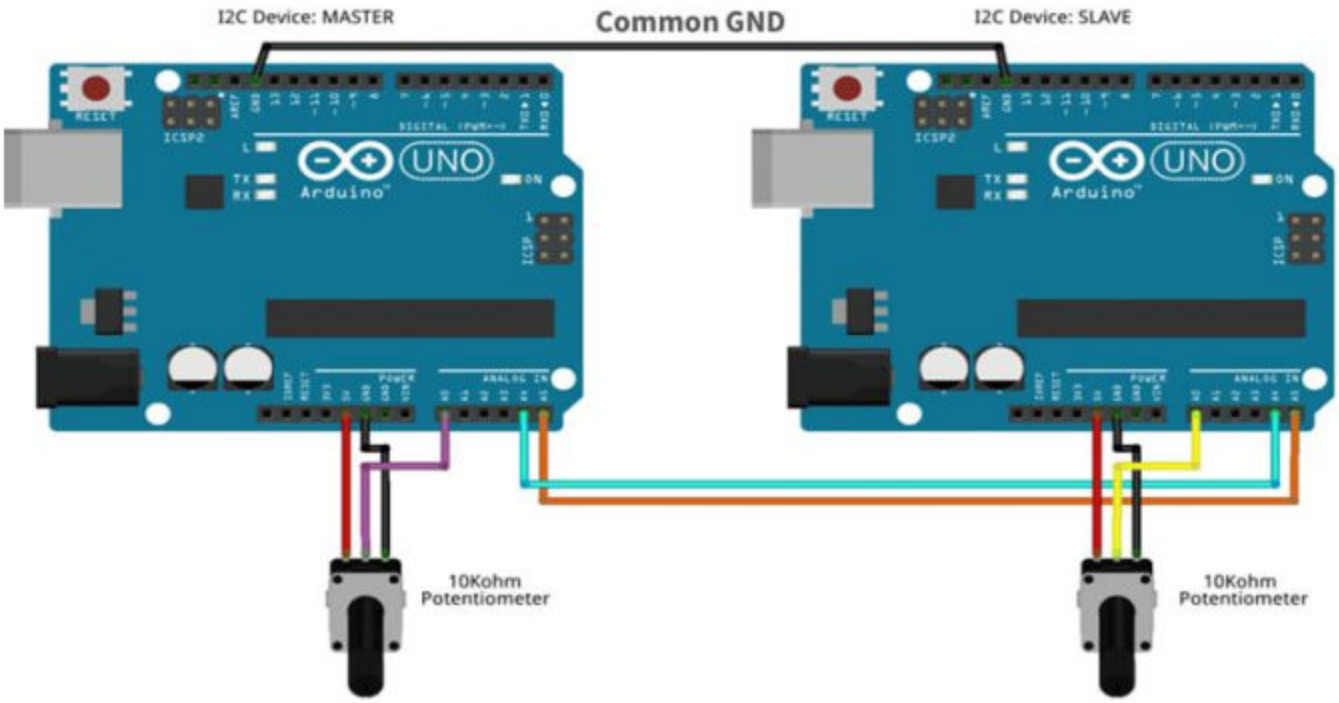
يوجد مقاومات رفع مدمجة مخصصة لهذه الأرجل (كما يتطلب بروتوكول I2C)



بعض تطبيقات I2C على المتحكمات الدقيقة :

التطبيق الأول : التواصل بين لوحتي أردوينو

سنقوم بهذا التطبيق بقراءة قيمة المقاومة المتغيرة المتصلة بلوحة الأردوينو الأساسية (Master) و إرسالها إلى لوحة الأردوينو التابعة (Slave) عبر بروتوكول I2C لتغيير معدل وميض LED على لوحة الأردوينو التابعة ، والعكس صحيح ..



الكود الخاص بلوحة الأردوينو القائدة Master :

```
#include <Wire.h>                    // تضمين مكتبة Wire الخاصة باتصال I2C
byte i2c_rcv;                        // متغير لتخزين البيانات المستقبلة من ناقل I2C
unsigned long time_start;            // متغير لتخزين وقت بداية عداد الزمن بالميلي ثانية
int stat_LED;                        // متغير لحالة الـ LED
byte value_pot;                      // متغير لتخزين قيمة قراءة المقاومة المتغيرة
void setup()
{
  Wire.begin();                      // تهيئة التحكم كـ Master على ناقل I2C
  i2c_rcv = 255;                     // إعطاء قيمة ابتدائية للقيمة المستلمة
  time_start = millis();             // حفظ الزمن الحالي كبداية لحساب زمن الوميض
  stat_LED = 0;                      // جعل حالة LED في البداية مطفأة
  pinMode(13, OUTPUT);               // ضبط الطرف 13 كمخرج لتوصيل LED
}
void loop()
{
  value_pot = analogRead(A0);        // قراءة قيمة الجهد على الطرف A0 (من 0 إلى 1023)
  Wire.beginTransmission(0x08);      // بدء إرسال بيانات إلى الجهاز Slave صاحب العنوان 0x08
  Wire.write(value_pot);             // إرسال قيمة المقاومة المتغيرة عبر I2C إلى الـ Slave
  Wire.endTransmission();            // إنهاء عملية الإرسال وإخبار الناقل بانتهاء البيانات
  Wire.requestFrom(0x08, 1);         // طلب بايت واحد من الجهاز Slave عنوانه 0x08
  if (Wire.available()) {            // التأكد من وجود بيانات مستلمة من الـ Slave
    i2c_rcv = Wire.read();           // قراءة البايت القادم من الـ Slave وتخزينه
  }
 // ---------------------------------------------
 // كود التحكم بسرعة وميض LED حسب القيمة المستقبلة
 // ---------------------------------------------
```

```
  if ((millis() - time_start) > (1000 * (float)(i2c_rcv / 255)))          // مقارنة الزمن المنقضي مع الزمن المطلوب للوميض
 {
   stat_LED = !stat_LED;          // LED عكس حالة
   time_start = millis();         // إعادة تسجيل الزمن الحالي كبداية دورة جديدة
  }
  digitalWrite(13, stat_LED);    // stat_LED حسب قيمة LED تشغيل أو إطفاء
}
```

الكود الخاص بلوحة الأردوينو التابعة Slave :

```
#include <Wire.h>              // I2C الخاصة باتصال Wire تضمين مكتبة
byte i2c_rcv;                  // I2C عبر Master متغير لتخزين البيانات المستلمة من الـ
unsigned long time_start;      // متغير لحفظ زمن بداية حساب فترة الوميض بالميلي ثانية
int stat_LED;                  // LED متغير لحالة الـ
byte value_pot;                // متغير لحفظ قيمة قراءة المقاومة المتغيرة
void setup()
{
  Wire.begin(0x08);            // 0x08 بعنوان I2C على ناقل Slave تشغيل التحكم كـ
  Wire.onReceive(dataRcv);     // Master تسجيل دالة تُنفذ تلقائياً عند وصول بيانات من الـ
  Wire.onRequest(dataRqst);    // Master تسجيل دالة تُنفذ تلقائياً عندما يطلب الـ بيانات
  i2c_rcv = 255;               // إعطاء قيمة ابتدائية للقيمة المستلمة
  time_start = millis();       // تخزين الزمن الحالي كبداية لحساب زمن الوميض
  stat_LED = 0;                // LED جعل LED في البداية مطفأ
  pinMode(13, OUTPUT);         // LED كخرج لتوصيل الـ 13 ضبط الطرف
}
void loop()
 {
```

```
  value_pot = analogRead(A0);          // قراءة قيمة الجهد على الطرف A0 من المقاومة المتغيرة

 // ---------------------------------------------

 // كود التحكم بسرعة وميض LED حسب القيمة المستلمة

 // ---------------------------------------------

  if ((millis() - time_start) > (1000 * (float)(i2c_rcv / 255)))          // مقارنة الزمن المنقضي مع زمن الوميض المطلوب

 {

    stat_LED = !stat_LED;                  // عكس حالة LED

    time_start = millis();                 // إعادة ضبط عداد الزمن من اللحظة الحالية

  }

 digitalWrite(13, stat_LED);           // تشغيل أو إطفاء LED حسب قيمة stat_LED

}

// ---------------------------------------------

// دالة استقبال البيانات من الـ Master

// ---------------------------------------------

void dataRcv(int numBytes)          // تستدعى تلقائياً عند وصول بيانات عبر I2C

 // numBytes يمثل عدد البايتات المستلمة

{

  while (Wire.available()) {          // التأكد من وجود بيانات متاحة

   i2c_rcv = Wire.read();             // قراءة البيانات القادمة من الـ Master وتخزينها

  }

}

// ---------------------------------------------

// دالة إرسال البيانات إلى الـ Master

// ---------------------------------------------

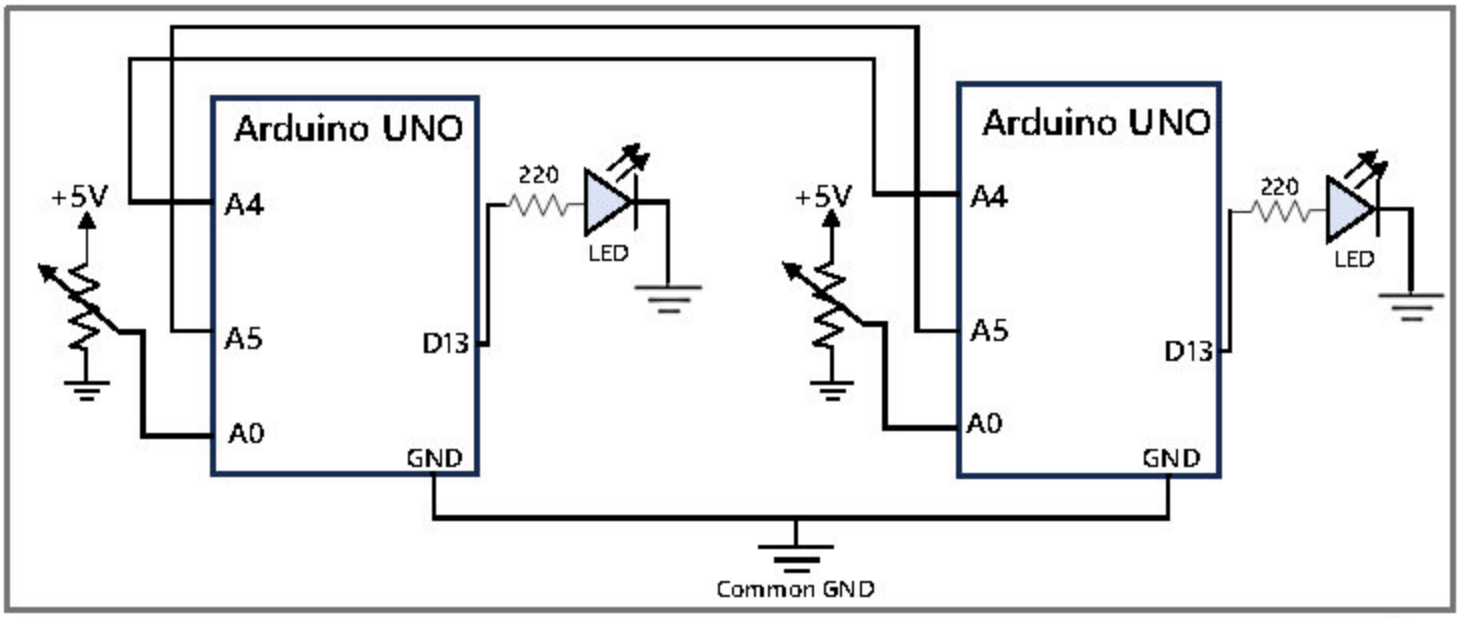
void dataRqst()                  // تستدعى تلقائياً عندما يطلب الـ Master بيانات

{

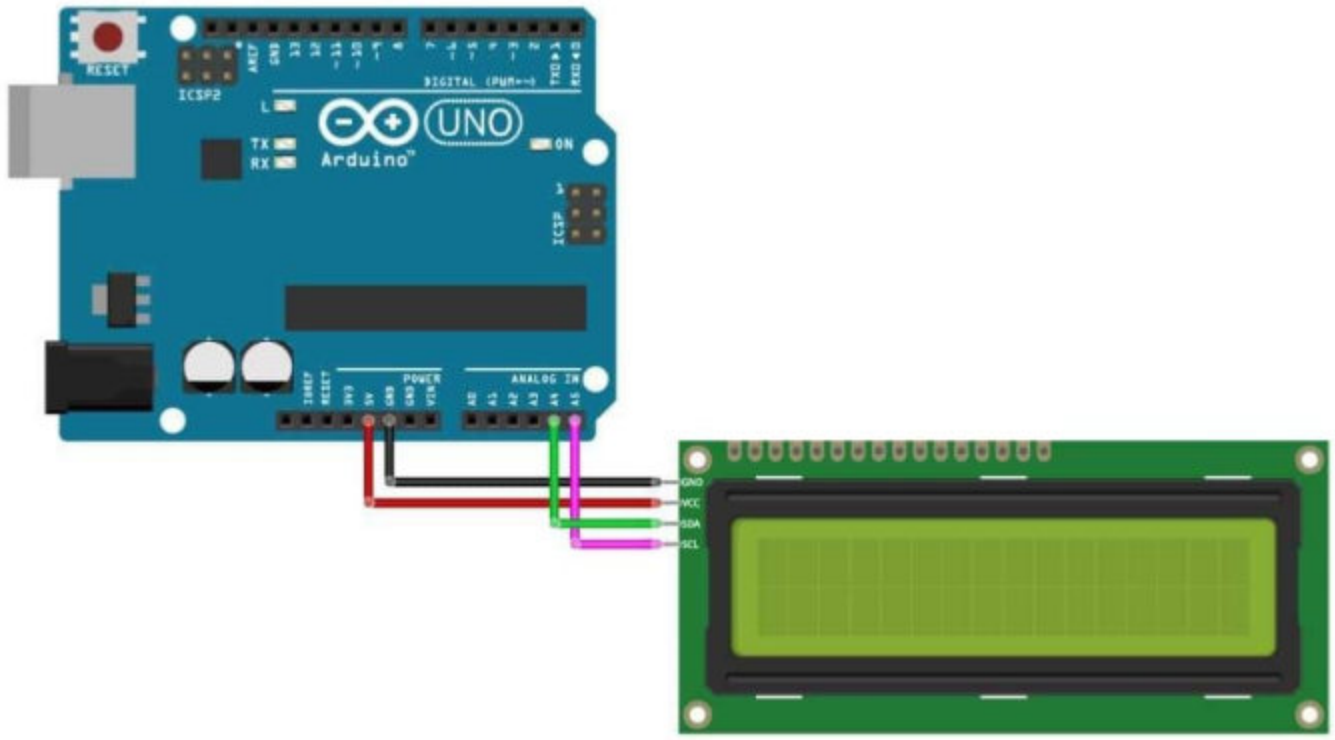
 Wire.write(value_pot);          // إرسال قيمة المقاومة المتغيرة إلى الـ Master

}
```





التطبيق الثاني : تشغيل شاشة LCD عبر بروتوكول I2C

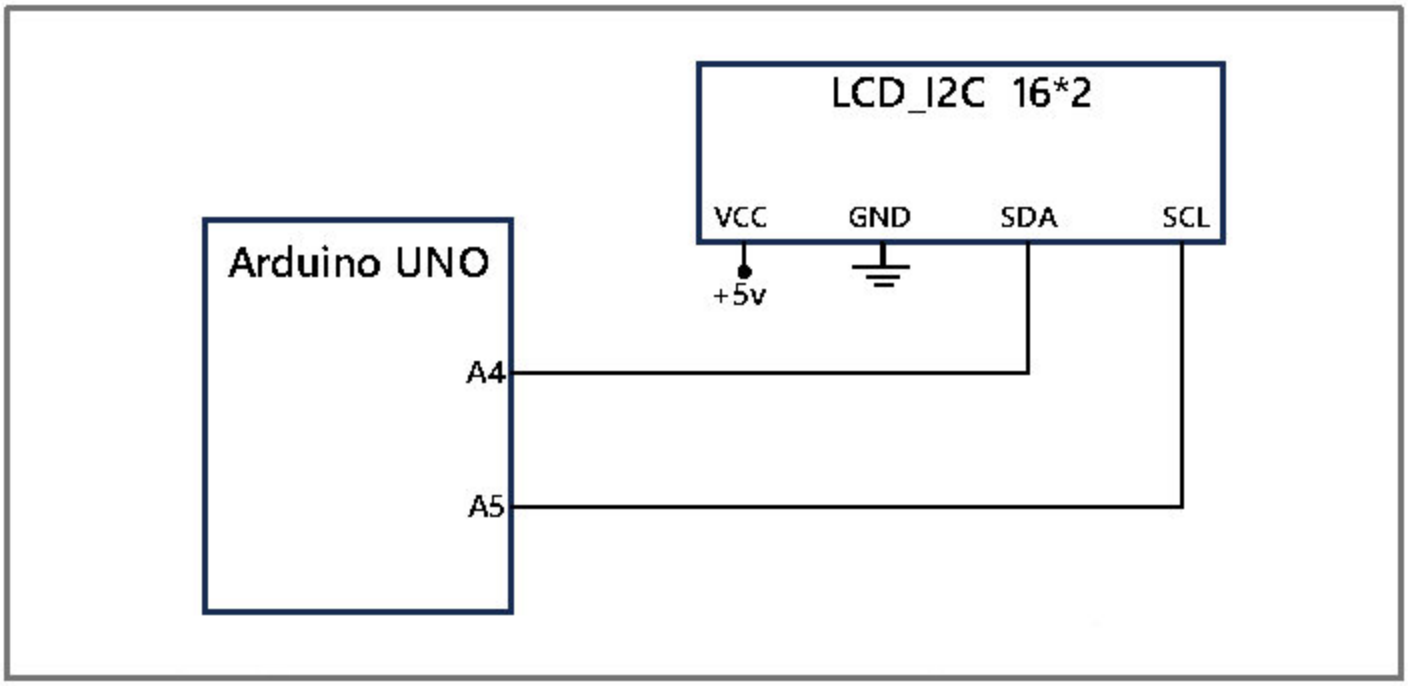


```
#include <LiquidCrystal_I2C.h>              // تضمين مكتبة التحكم بشاشة LCD عبر بروتوكول I2C

LiquidCrystal_I2C lcd(0x27, 16, 2);         // إنشاء كائن للشاشة بعنوان 0x27

void setup()
{
  lcd.init();                               // تهيئة شاشة LCD وبدء الاتصال معها
  lcd.backlight();                          // تشغيل الإضاءة الخلفية للشاشة
```

```
}
void loop() {
  lcd.clear();                  // مسح محتويات الشاشة الحالية
  lcd.setCursor(0, 0);          // تحديد مكان الكتابة عند العمود 0 والصف 0
  lcd.print("Arduino");         // طباعة كلمة Arduino على الشاشة
  lcd.setCursor(0, 1);          // نقل مؤشر الكتابة إلى العمود 0 والصف 1
  lcd.print("I2C LCD");         // طباعة عبارة I2C LCD على الشاشة
  delay(1000);                  // انتظار ثانية واحدة قبل إعادة تنفيذ الحلقة
}
```



أعِنَّا علينَا لنسعَى إليك.

بروتوكول SPI

- يرمز الاسم إلى Serial Peripherals Interface .
- مصمم خصيصاً للمتحكمات الدقيقة .
- يعمل بوضع Full-Duplex .
- يعمل بمعدلات نقل بيانات تصل لـ 8 Mbps و أكثر .
- عادة ما يكون أسرع من I2C لأنه على الرغم من أن خطوط البيانات و الساعة مشتركة بين الأجهزة إلا أن كل جهاز تابع له خط اختيار مستقل (CS) لتحديده أثناء الاتصال بدلاً من استخدام العناوين كما في I2C .
- يُستخدم SPI في التطبيقات التي تتطلب سرعة عالية في نقل البيانات مثل: (بطاقات SD – شاشات العرض – الأنظمة التي تتغير فيها البيانات بسرعة و تتطلب تحديثاً فورياً) .

كيف يعمل بروتوكول SPI ؟

1. يمكن التواصل عبر SPI بطريقتين رئيسيتين:

   - اختيار كل جهاز عبر خط Chip Select (CS):

في هذه الطريقة يتم توصيل خط تحديد مستقل لكل جهاز طرفي، حيث يقوم المتحكم الرئيسي بتفعيل خط CS الخاص بالجهاز المطلوب فقط للتواصل معه، بينما تبقى بقية الأجهزة غير مفعلة. وبالتالي يحتاج كل جهاز إلى خط اختيار خاص به.

   - التوصيل المتسلسل (Daisy Chain):

في هذا الأسلوب يتم توصيل الأجهزة بشكل متسلسل، حيث يكون خرج البيانات (Data Out) من جهاز ما متصلًا بمدخل البيانات (Data In) للجهاز الذي يليه، مما يسمح بتمرير البيانات عبر سلسلة الأجهزة.

2. لا يوجد حد نظري صارم لعدد أجهزة SPI التي يمكن توصيلها، لكن عمليًا يتم تحديد العدد بناءً على:

✓ 1 عدد خطوط Chip Select المتاحة في المتحكم الرئيسي عند استخدام طريقة الاختيار المباشر

✓ 2 أو تعقيد تمرير البيانات بين الأجهزة عند استخدام أسلوب التوصيل المتسلسل

3. في حالة الاتصال من نقطة إلى نقطة (Point-to-Point)، لا يحتاج SPI إلى نظام عناوين (Addressing)، كما أنه يدعم الاتصال ثنائي الاتجاه (Full-Duplex)، مما يجعله بسيطًا وفعّالًا في نقل البيانات.

4. يستخدم بروتوكول Serial Peripheral Interface أربعة خطوط اتصال رئيسية :

MOSI (Master Out Slave In)

خط يُستخدم لإرسال البيانات من الجهاز الرئيسي (Master) إلى الجهاز التابع (Slave)، وفي نفس الوقت يكون مدخل بيانات بالنسبة للـ Slave.

MISO (Master In Slave Out)

خط يُستخدم لإرسال البيانات من الجهاز التابع إلى الجهاز الرئيسي، أي هو مدخل بيانات للـ Master ومخرج للـ Slave.

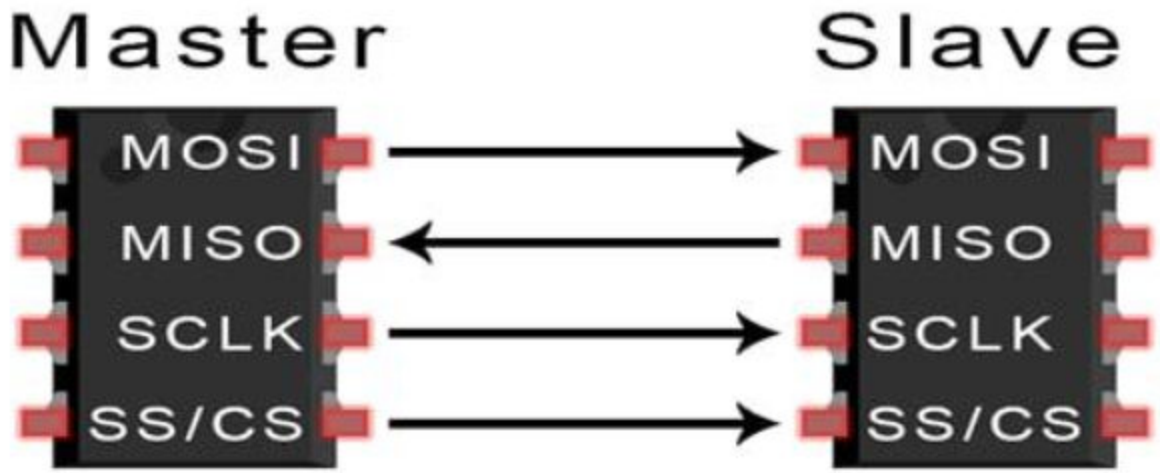
SCLK (Serial Clock)

إشارة الساعة التي يولدها الجهاز الرئيسي لتنظيم عملية نقل البيانات وتزامنها. يتم تحديد تردد هذه الإشارة من خلال مقسّم تردد داخلي (Clock Divider)، والقيمة الافتراضية غالبًا تكون SPI_CLOCK_DIV4، مع خيارات تقسيم مثل 2، 4، 8، 16، 32، 64، أو 128.

SS (Slave Select) أو CS (Chip Select)

إشارة تُستخدم لتفعيل الجهاز التابع المطلوب فقط، ويقوم المتحكم الرئيسي بالتحكم بها. عند تفعيلها يصبح الجهاز قادرًا على التواصل، وعند تعطيلها يتم عزله عن Bus.

في الأنظمة التي تحتوي على أكثر من جهاز تابع، يحتاج كل Slave إلى خط CS مستقل، وهذا يجعل التوصيل أكثر تعقيدًا من بروتوكول I2C من ناحية العتاد.



5. يتكوّن واجه SPI في الواقع من مسجلّي إزاحة (Shift Registers) بسيطين داخل العتاد الداخلي، حيث يتم تبادل البيانات بين الطرفين بطريقة متسلسلة بتًّا تلو الآخر.

عادةً ما تكون البيانات المنقولة بطول 8 بت، ويتم إرسالها بتسلسل bit-by-bit أثناء تفعيل إشارة اختيار الجهاز التابع (Slave Enable)، وبالتزامن مع نبضات الساعة (Shift Clock) التي يولدها الجهاز الرئيسي.

6. يُعتبر بروتوكول SPI وسيلة نقل بيانات تسلسلية متزامنة (Synchronous Serial Communication) بين وحدة المعالجة المركزية (CPU) والأجهزة الطرفية ذات السرعة المنخفضة نسبيًا.

7. وهو يدعم الاتصال ثنائي الاتجاه الكامل (Full-Duplex)، أي يمكن الإرسال والاستقبال في نفس الوقت.كما أن سرعة نقل البيانات فيه تكون عادة أعلى من بروتوكول I2C، ويمكن أن تصل إلى عدة ميجابت في الثانية (Mbps)، مما يجعله مناسبًا للتطبيقات التي تتطلب أداءً أسرع .

مزايا استخدام بروتوكول Serial Peripheral Interface:

- ✓ يتميز البروتوكول بالبساطة، لأنه لا يحتوي على نظام معقد لتحديد العناوين (Slave Addressing) كما في I2C
- ✓ يُعتبر أسرع بروتوكول مقارنةً بـ UART و I2C من حيث سرعة نقل البيانات.
- ✓ لا يحتوي على بتات بدء وإيقاف (Start/Stop bits) مثل UART، مما يسمح بإرسال البيانات بشكل مستمر دون انقطاع.
- ✓ يعتمد على خطوط منفصلة للإرسال والاستقبال (MISO و MOSI)، مما يتيح إمكانية الإرسال والاستقبال في نفس الوقت (اتصال ثنائي الاتجاه الكامل Full Duplex).

عيوب استخدام بروتوكول SPI:

- يستهلك عددًا أكبر من المنافذ (Pins)، مما يحد عمليًا من عدد الأجهزة التي يمكن توصيلها على نفس النظام.

- لا يحتوي على آلية للتحكم بتدفق البيانات (Flow Control)، كما أنه لا يوفر آلية تأكيد استلام البيانات (Acknowledgment) مثل الموجودة في I2C، لذلك لا يمكن التأكد من وصول البيانات بشكل مباشر.

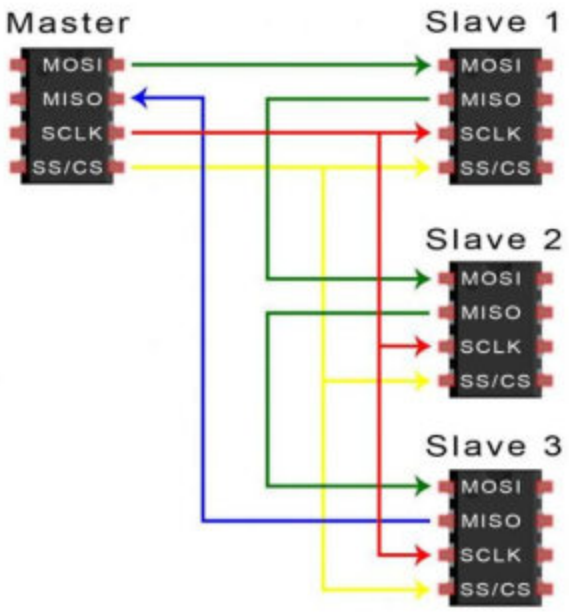
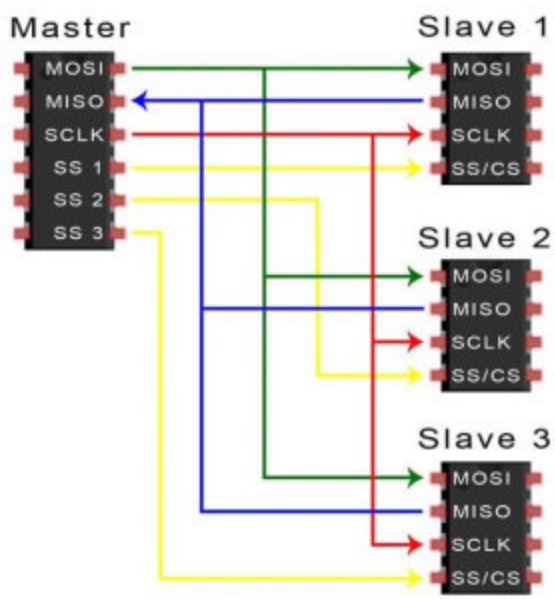
- يستخدم أربع خطوط اتصال رئيسية:

  MOSI , MISO , SCLK , SS

- لا يحتوي على أي آلية مدمجة لاكتشاف الأخطاء (Error Checking)، على عكس UART الذي يمكن أن يستخدم بت التماثل (Parity Bit).

- يدعم وجود متحكم رئيسي واحد فقط (Single Master) في النظام.

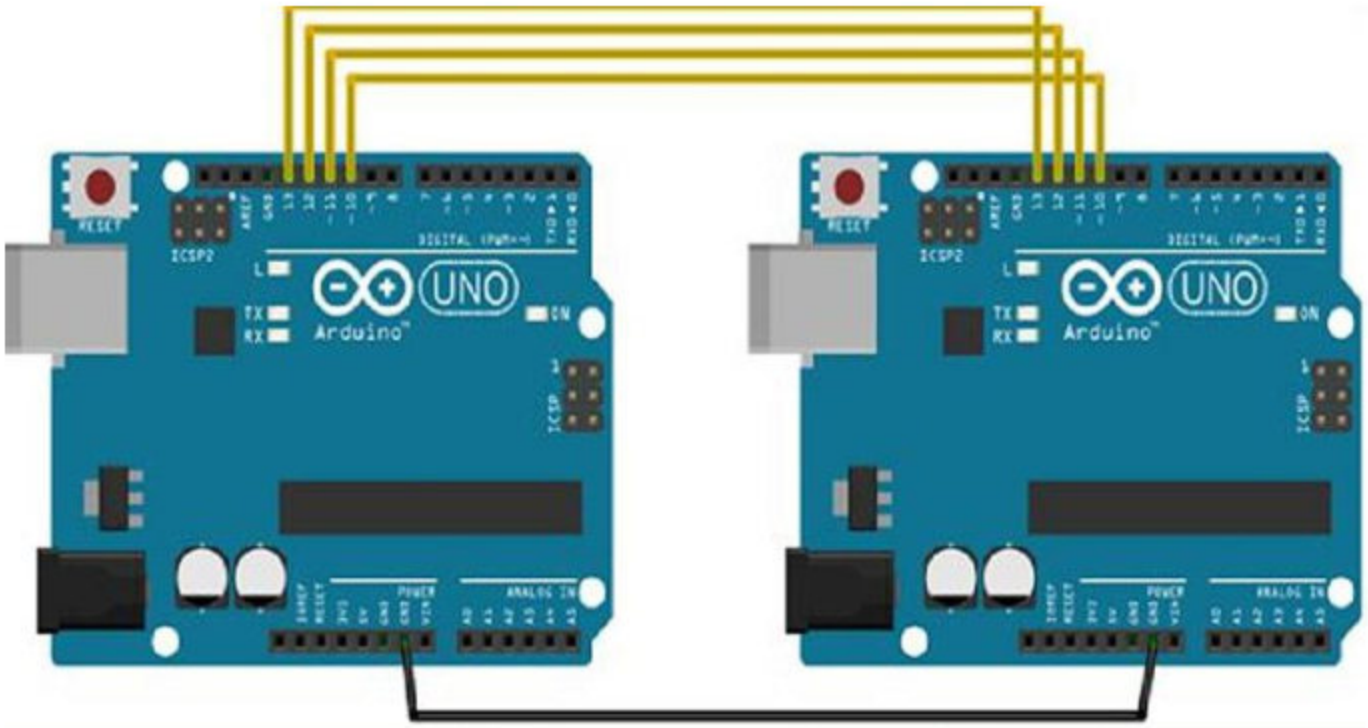
طرق توصيل الأجهزة ببعضها عبر بروتوكول SPI :

التوصيل المتسلسل	اختيار كل جهاز عبر خط Chip select
	

الأرجل الخاصة ببروتوكول SPI في لوحة Arduino UNO :

- SS Pin 10
- MOSI Pin 11
- MISO Pin 12
- SCKL Pin 13

تطبيق التواصل بين لوحتي أردوينو :



الكود الخاص بلوحة الأردوينو القائدة Master :

```
#include <SPI.h>                          // استدعاء مكتبة SPI لتفعيل بروتوكول الاتصال التسلسلي السريع //

void setup()
{
 Serial.begin(115200);                    // تهيئة الاتصال التسلسلي لعرض البيانات على Serial Monitor بسرعة 115200 //
 digitalWrite(SS, HIGH);                  // جعل خط SS (Slave Select / Chip Select) في الحالة HIGH لتعطيل الجهاز التابع افتراضياً //
 SPI.begin();                             // بدء تهيئة بروتوكول SPI وتفعيل وضع Master //
 SPI.setClockDivider(SPI_CLOCK_DIV8);     // تقليل سرعة ساعة SPI إلى 1/8 من سرعة المعالج لضبط سرعة الإرسال //
}
void loop()
{
 char c;
 digitalWrite(SS, LOW);                   // تفعيل الجهاز التابع (Slave) عبر جعل SS = LOW //

      /*  SPI  إرسال سلسلة نصية تجريبية عبر */

 for (const char *p = "Hello, world!\r"; c = *p; p++)
 {
```



طيب ليش استخدمنا حلقة for ؟
وكيف رح تشتغل هوون؟؟

ما بعرف كيف اشرحلك ياها بس
لو بهمك كنت عرفت لحالك !

```
  SPI.transfer(c);                    // إرسال حرف واحد عبر SPI واستقبال أي بيانات واردة بنفس اللحظة (Full Duplex)
  Serial.print(c);                    // طباعة الحرف المرسل على Serial Monitor للمراقبة
 }
 digitalWrite(SS, HIGH);              // تعطيل الجهاز التابع بعد انتهاء الإرسال
 delay(1000);                         // تأخير 1 ثانية قبل تكرار الإرسال
}
```

1ـ الجزء الأول:

`const char * p = "Hello, world!\r"`

هنا يتم إنشاء مؤشر اسمه p يشير إلى بداية النص، أي أنه يشير إلى أول حرف وهو (H) وكلما تحرك المؤشر ينتقل إلى الحرف الذي يليه.

2ـ الجزء الثاني (شرط استمرار الحلقة):

`c = *p`

يعني: خذ الحرف الذي يشير إليه المؤشر p وضعه داخل المتغير c

مثلاً في أول مرة:

p يشير إلى H إذن c = H

و تستمر الحلقة بالعمل طالما أن قيمة c ليست صفراً ...

عندما يصل المؤشر إلى نهاية النص ويقرأ الرمز { \0 } وهو رمز نهاية السلسلة النصية ، تتوقف الحلقة.

3ـ الجزء الثالث:

`p++`

بعد تنفيذ أوامر الحلقة ، يتم زيادة المؤشر p للانتقال إلى الحرف التالي.

أي : في الدورة الأولى مثلاً : p → H

و بعد : p++

p → e

ثم:

p → l

وهكذا حتى نهاية الجملة ...

4ـ جسم الحلقة:

`SPI.transfer(c); Serial.print(c);`

في كل دورة يتم:

- ✓ إرسال الحرف الموجود في c عبر بروتوكول SPI
- ✓ طباعة نفس الحرف على Serial Monitor



باختصار:

حلقة for وظيفتها قراءة النص "Hello, world!\r" حرفاً حرفاً، ثم إرسال كل حرف عن طريق SPI وطباعته على Serial Monitor إذ تمر الحلقة على كل حرف في النص باستخدام المؤشر p ، تخزن الحرف في المتغير c ، ترسله عبر SPI، ثم تنتقل للحرف التالي حتى تصل إلى نهاية النص.

الكود الخاص بلوحة الأردوينو التابعة Slave :

```
#include <SPI.h>                    // استدعاء مكتبة SPI لتفعيل بروتوكول الاتصال التسلسلي السريع
char buff[50];                      // مصفوفة (Buffer) لتخزين البيانات المستقبلة عبر SPI بحد أقصى 50 حرف
volatile byte indx;                 // مؤشر (Index) متغير يُستخدم لتحديد مكان التخزين داخل المصفوفة
volatile boolean process;           // متغير علم (Flag) لتحديد متى تكتمل عملية استقبال البيانات
void setup()
{
 Serial.begin(115200);              // تهيئة الاتصال التسلسلي لعرض البيانات على Serial Monitor
 pinMode(MISO, OUTPUT);             // ضبط خط MISO كمخرج لأن الجهاز هنا يعمل كـ Slave ويرسل البيانات إلى Master
 SPCR |= _BV(SPE);                  // تفعيل SPI في وضع Slave عبر سجل التحكم SPI Control Register (SPE = SPI Enable)
 indx = 0;                          // تهيئة المؤشر ليبدأ من أول المصفوفة
 process = false;                   // لا توجد بيانات جاهزة للمعالجة في البداية
 SPI.attachInterrupt();             // تفعيل مقاطعة SPI عند اكتمال كل عملية إرسال/استقبال
}
ISR(SPI_STC_vect)                   // جسم المقاطعة الخاص بـ SPI
{
 byte c = SPDR;                     // قراءة البايت المستلم من سجل بيانات SPI
 if (indx < sizeof(buff))           // التأكد من عدم تجاوز حجم المصفوفة (منع overflow)
 {
  buff[indx++] = c;                 // تخزين البايت في أول خانة فارغة داخل buffer ثم زيادة المؤشر
 }
 if (c == '\r')                     // التحقق إذا كان الحرف المستلم هو رمز نهاية الرسالة
 {
  process = true;                   // تفعيل علم (Flag) لإخبار البرنامج الرئيسي أن الرسالة اكتملت
 }
}
void loop()
{
```



```
if (process)
{
  process = false;                          // إعادة ضبط العلم بعد بدء المعالجة
  Serial.println(buff);                     // طباعة النص الكامل المستلم على Serial Monitor
  indx = 0;                                 // إعادة المؤشر إلى البداية لاستقبال رسالة جديدة
 }
}
```

أي بروتوكول اتصال هو الأفضل UART أم SPI أم I2C ؟



للأسف، لا يوجد بروتوكول اتصال يمكن اعتباره "الأفضل" بشكل مطلق.
كل بروتوكول من بروتوكولات الاتصال التسلسلي له مزاياه وعيوبه الخاصة.

لذلك يجب على المستخدم اختيار بروتوكول الاتصال الذي يناسب مشروعه بشكل أفضل.

فعلى سبيل المثال:

- **إذا كان الهدف هو الحصول على أكبر سرعة نقل بيانات، فإن بروتوكول SPI هو الخيار المثالي.**
- **أما إذا كان الهدف هو توصيل عدد كبير من الأجهزة مع بساطة في التوصيل والإدارة، فإن بروتوكول I2Cايكون الخيار الأفضل، لأنه يسمح بتوصيل ما يصل إلى 128جهازًا ويتميز بسهولة التنظيم.**

على أي حال، يجب الانتباه إلى أن جميع الأجهزة أو الحساسات أو الوحدات المستخدمة يجب أن تدعم بروتوكول الاتصال المختار حتى يمكن استخدامها بشكل صحيح.

الجدول الآتي يوضح الفروقات بشكل عام بين البروتوكولات التي تعاملنا معها إلى الآن :

Protocol	UART	I2C	SPI
Complexity	Simple	Easy to chain multiple devices	Complex as device increases
Speed	Slowest	Faster than UART	Fastest
Number of devices	Up to 2 devices	Up to 127, but gets complex	Many, but gets complex
Number of wires	1	2	4
Duplex Mode	Full Duplex	Half Duplex	Full Duplex
No. of masters and slaves	Single to Single	Multiple slaves and masters	1 master, multiple slaves

بروتوكول CAN Bus :

- هو اختصار لعبارة Control Area Network أي شبكة منطقة التحكم.
- تم تطوير بروتوكول CAN في الأصل بواسطة شركة Bosch.
- يتيح ربط عدة أجهزة معًا والتواصل بينها في الوقت نفسه عبر ناقل اتصال واحد.
- يُعد CAN Bus معيارًا لناقل اتصال استُخدم في البداية داخل السيارات، حيث صُمم لتمكين المتحكمات الدقيقة والأجهزة الإلكترونية من تبادل البيانات فيما بينها دون الحاجة إلى حاسب مركزي (Host Computer). ويعتمد على تبادل الرسائل (Message-Based Protocol) بدلاً من إرسال البيانات بين عناوين محددة. وقد كان الهدف الأساسي من تطويره هو تقليل كمية الأسلاك الكهربائية داخل السيارات، وبالتالي خفض استهلاك النحاس، إلا أنه أصبح يُستخدم اليوم في العديد من التطبيقات الأخرى أيضًا.

- يتميز بروتوكول CAN بقدرته العالية على العمل في البيئات التي تحتوي على تشويش كهرومغناطيسي مرتفع، كما يمتلك آليات فعّالة لاكتشاف الأخطاء وتصحيحها، مما يجعله بروتوكولًا يتمتع بموثوقية عالية. لذلك يُستخدم غالبًا في التطبيقات الحساسة والحرجة التي تتطلب اعتمادية كبيرة.

- بالمقارنة مع بروتوكولات SPIوI2C وUART، يستطيع CAN نقل البيانات عبر مسافات أطول بكثير مع الحفاظ على موثوقية الاتصال.

- يستخدم بروتوكول CAN بنية (Multi-Master)، أي يمكن لأي جهاز متصل بالناقل أن يبدأ بإرسال البيانات دون وجود جهاز رئيسي واحد يتحكم بعملية الاتصال. كما أن جميع الرسائل تُبث إلى جميع الأجهزة المتصلة بالشبكة. ويعتمد CAN على الإشارة التفاضلية (Differential Signal)، التي تستخدم حالتين منطقيتين هما:

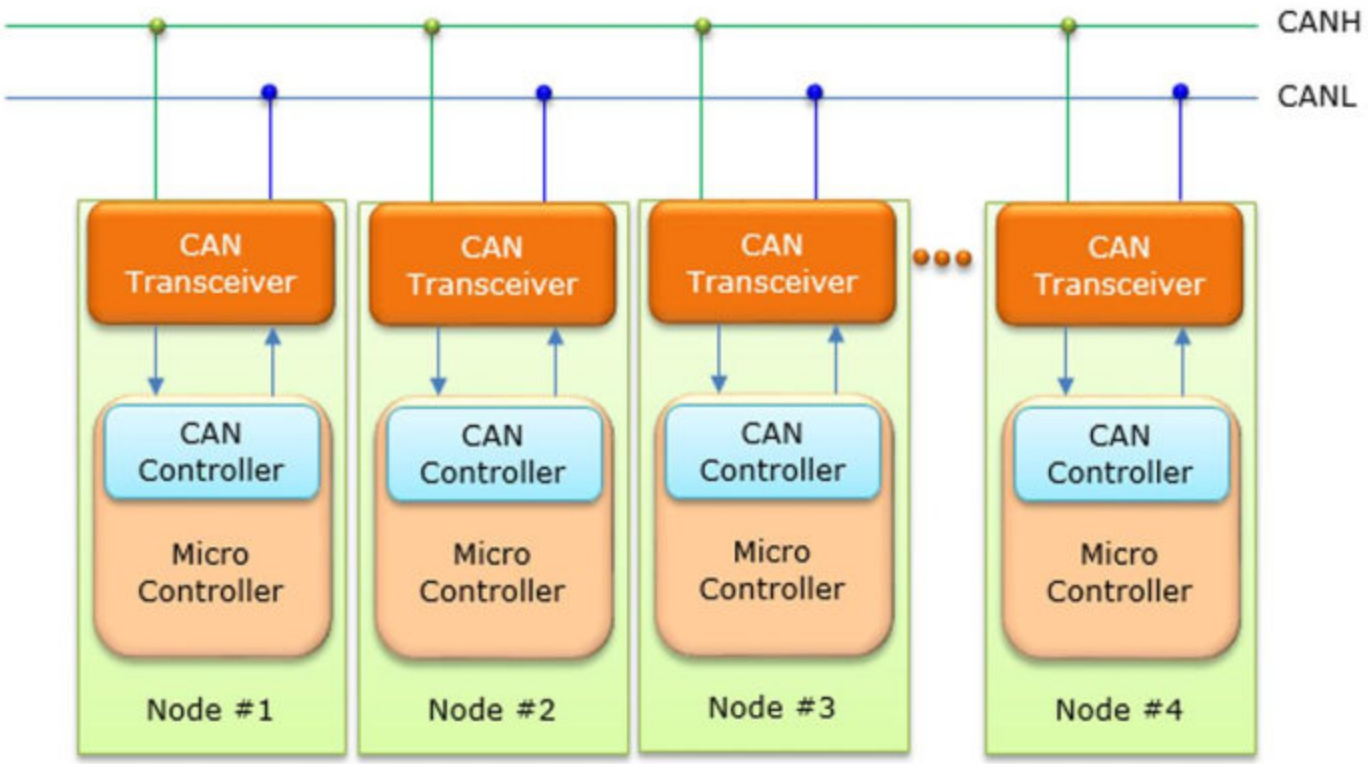
  - Dominant (0): الحالة المسيطرة.

  - Recessive (1): الحالة غير المسيطرة.

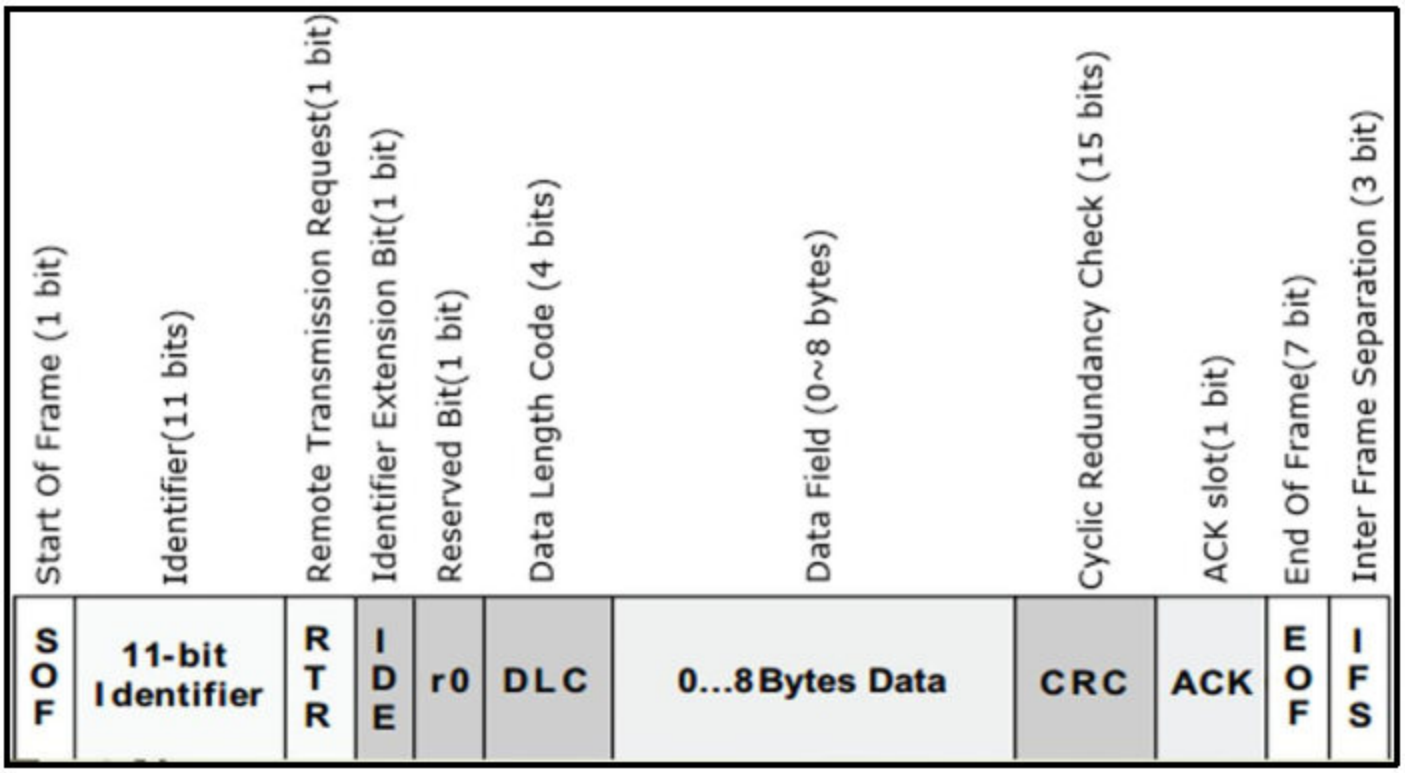
- يعتمد CAN على بروتوكول CSMA/CD، أي أن كل عقدة (Node) على الناقل تستطيع اكتشاف حدوث تصادم (Collision) عند محاولة أكثر من جهاز الإرسال في الوقت نفسه. وعند حدوث ذلك، تتوقف العقدة عن الإرسال لفترة زمنية محددة ثم تعيد المحاولة. ويتم حل التصادم من خلال آلية تحكيم تعتمد على أولوية معرف الرسالة (Message Identifier)، بحيث تستمر الرسالة ذات الأولوية الأعلى في الإرسال بينما تنتظر الرسائل الأخرى.

- لا يحتوي إطار البيانات في CAN على حقل عنوان (Address Field)، لذلك لا تكون الرسالة موجهة إلى جهاز محدد، بل تُرسل إلى جميع الأجهزة المتصلة بالناقل (Bus). أي أن CAN يعتمد أساسًا على أسلوب البث (Broadcast)، حيث تستقبل جميع الأجهزة الرسالة نفسها، ثم يقرر كل جهاز بنفسه ما إذا كانت الرسالة تخصه فيقوم بمعالجتها، أو يتجاهلها إذا لم تكن ذات صلة به.

رسم توضيحي لشبكة CAN :



شكل حزمة البيانات في بروتوكول CAN :



مزايا استخدام بروتوكول CAN :

✓ **انخفاض التكلفة:** عند تطوير بروتوكول CAN لأول مرة، كان الهدف الأساسي هو توفير وسيلة اتصال سريعة بين الوحدات والأجهزة الإلكترونية داخل المركبات، مع تقليل كمية الأسلاك المستخدمة، مما أدى إلى خفض التكلفة وتقليل الوزن.

✓ **اكتشاف الأخطاء المدمج:** يحتوي بروتوكول CAN على آليات مدمجة لاكتشاف الأخطاء أثناء نقل البيانات، مثل:

- خطأ البت (Bit Error).
- خطأ التأكيد (ACK Error).
- خطأ تنسيق الإطار (Form Error).
- خطأ فحص التكرار الدوري (CRC Error).
- خطأ حشو البتات (Stuff Error).

✓ **الموثوقية العالية:** يتميز CAN بأداء موثوق حتى في البيئات التي تعاني من تشويش كهرومغناطيسي مرتفع، كما يمتلك آليات فعّالة لاكتشاف الأخطاء والتعامل معها، مما يجعله مناسبًا للتطبيقات التي تتطلب اعتمادية كبيرة.

✓ **السرعة:** يدعم بروتوكول CAN معدل نقل بيانات يصل إلى 1 ميجابت/ثانية (1 Mbps)، بينما يدعم الإصدار المطور CAN FD (Flexible Data Rate) سرعات أعلى من ذلك.

✓ **المرونة:** يتميز بروتوكول CAN بمرونته وإمكانية استخدامه في العديد من المجالات، مثل الأنظمة الطبية، وأنظمة الترفيه، والأجهزة المحمولة، بالإضافة إلى التطبيقات الصناعية والسيارات.

عيوب استخدام بروتوكول CAN :

- **محدودية عدد الأجهزة المتصلة:** بسبب الأحمال الكهربائية على الناقل (Bus)، فإن عدد الأجهزة أو العقد (Nodes) التي يمكن توصيلها بشبكة CAN محدود، ويبلغ 64 جهازًا كحد أقصى في التكوين القياسي.

- **محدودية طول كابل الاتصال:** يقتصر طول كابل الاتصال في شبكة CAN على حوالي 40 مترًا عند العمل بأقصى سرعة نقل، وهو طول كافٍ لمعظم التطبيقات، خاصة في السيارات والأنظمة الصناعية، لكنه قد لا يكون مناسبًا لبعض التطبيقات التي تتطلب الاتصال عبر مسافات أطول.

تم بعون الله الانتهاء من بحث بروتوكولات الاتصال...



اكتب ملاحظاتك ♡:

فريقنا يتمنى لكم دراسة موفقة